

MOBILE APPLICATION DEVELOPMENT

Shoaib Farooq

Department of Computer Science

Instructor Information

Instructor	Shoaib Farooq	E-mail	m.shoaib1050@gmail.com
 GitHub	https://github.com/SHOAIB1050		
 YouTube	https://www.youtube.com/c/pakithub		
 LinkedIn	https://www.linkedin.com/in/shoaib-farooq-b5b190105/		

Let's Start

Lecture # 8

OUTLINE

- **Widget Lifecycle in Flutter**
- **List Tile Widget**



DidUpdateWidget

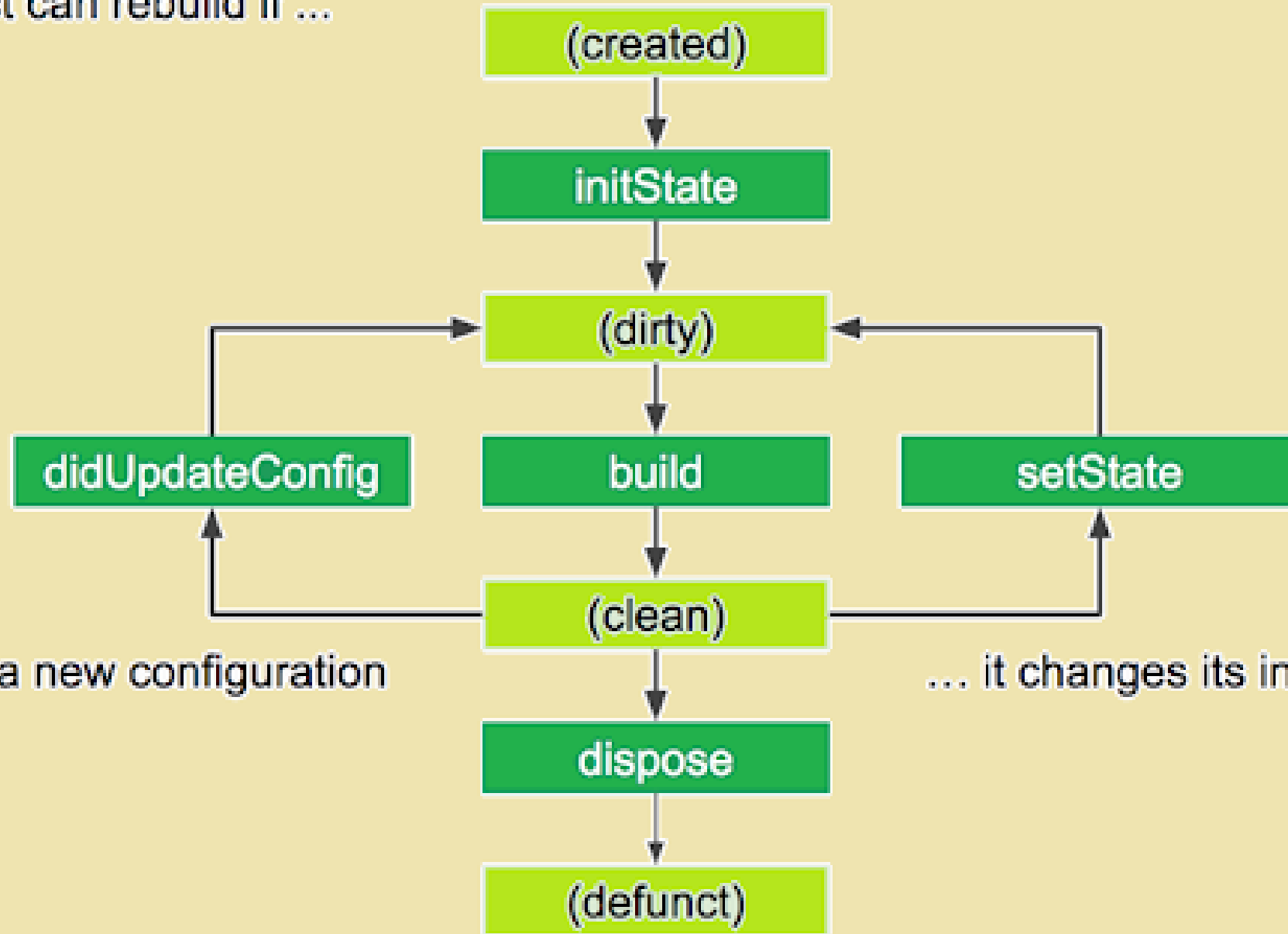
Dispose

InitState

Deactivate

DidChangeDependencies

A State<T> object can rebuild if ...



... it receives a new configuration

... it changes its internal state

initState()

Flutter's `initState()` method is the first method that is used while creating a stateful class, here we can initialize variables, data, properties, etc. for any widget.

```
int a;  
@override  
void initState() {  
    a = 0;  
    super.initState();  
}
```

```
25   @override
26   _MyHomePageState createState() => _MyHomePageState();
27 }
28
29 class _MyHomePageState extends State<MyHomePage> with SingleTickerProvider
30   AnimationController _animationController;
31
32   @override
33   void initState() {
34     super.initState();
35     // TODO: implement initState
36     print('initState')
37     _animationController = AnimationController(vsync: this);
38   }
39
40   @override
```


createState()

When we create a stateful widget, our framework calls a createState() method and it must be overridden.

```
class MyPage extends StatefulWidget {  
  @override  
  _MyPageState createState() => _MyScreenState();  
}
```

build():

The build method is used each time the widget is rebuilt. This can happen either after calling initState, didChangeDependencies, didUpdateWidget, or when the state is changed via a call to setState

```
@override  
Widget build(BuildContext context, MyButtonState state) {  
  return Container(color:Colors.red);  
}
```

didChangeDependencies()

This method is called immediately after initState and when dependency of the State object changes via InheritedWidget.

```
@override  
void didChangeDependencies() {  
    super.didChangeDependencies()  
}
```

```
28
29 class _MyHomePageState extends State<MyHomePage> with SingleTickerProviderStateMixin {
30   AnimationController _animationController;
31
32   @override
33   void initState() {
34     super.initState();
35     // TODO: implement initState
36     print('initState');
37     _animationController = AnimationController(vsync: this);
38   }
39
40   @override
41   void didChangeDependencies() {
42     // TODO: implement didChangeDependencies
43     print('didChangeDependencies');
44     super.didChangeDependencies();
45   }
46
47   @override
```

didUpdateWidget()

This method is called whenever the widget configuration changes. A typical case is when a parent passes some variable to the children() widget via the constructor.

```
@override  
void didUpdateWidget(MyHomePage oldWidget) {  
    super.didUpdateWidget(oldWidget)  
}
```

deactivate()

The deactivate() method is called when a widget is removed from the widget tree but hasn't yet been permanently disposed of. For instance, in situations where you have multiple screens and are navigating between them, the widget on the screen you're leaving will trigger deactivate()

```
@override  
void deactivate() {  
    super.deactivate();  
}
```

```
69 void didUpdateWidget(Covariant MyHomePage oldWidget) {  
70     super.didUpdateWidget(oldWidget);  
71     // TODO: implement didUpdateWidget  
72     print('didUpdateWidget');  
73     if (this.widget.title != oldWidget.title) {  
74         print('Title has changed');  
75     }  
76 }
```

```
78 @override  
79 void deactivate() {  
80     // TODO: implement deactivate  
81     print('deactivate');  
82     super.deactivate();  
83 }
```

```
86  
87 @override  
88 Widget build(BuildContext context) {  
89     return Scaffold(  
90         appBar: AppBar(  
91             title: Text(widget.title),  
92         ),  
93     );  
94 }
```

dispose()

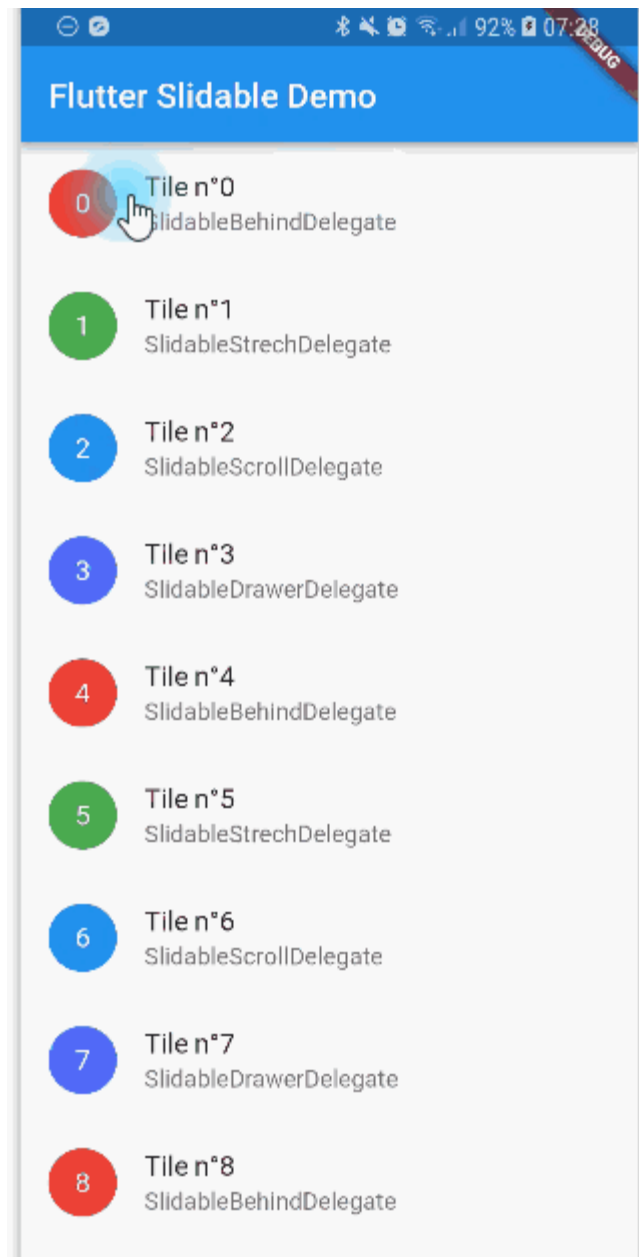
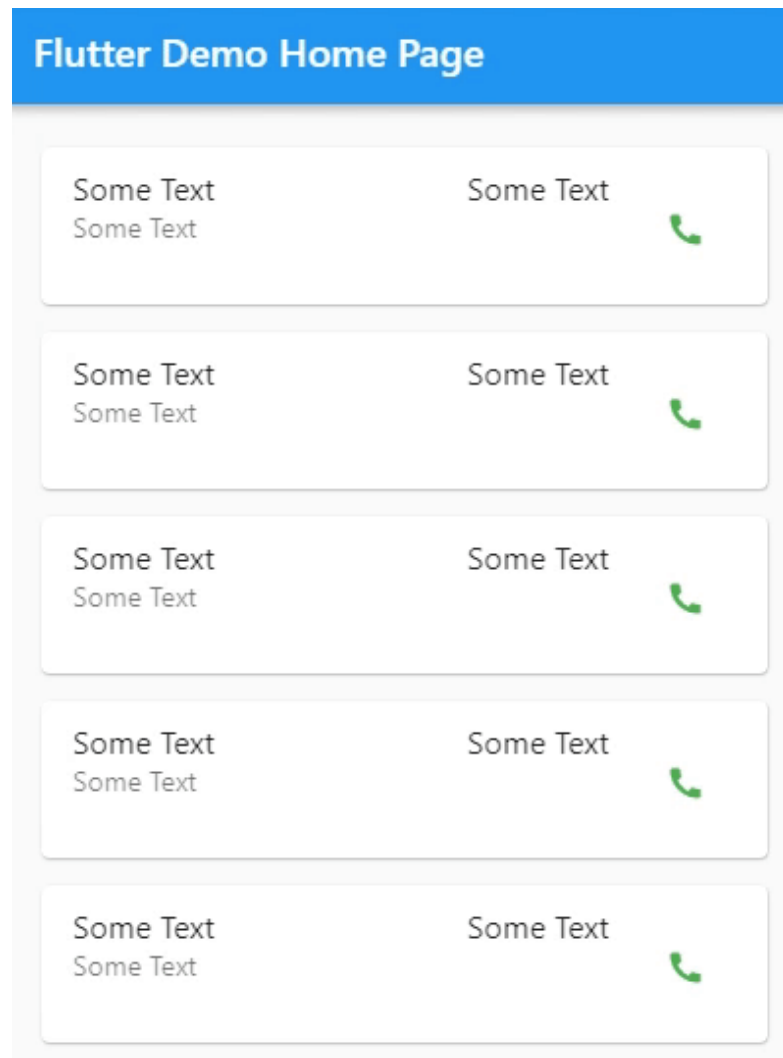
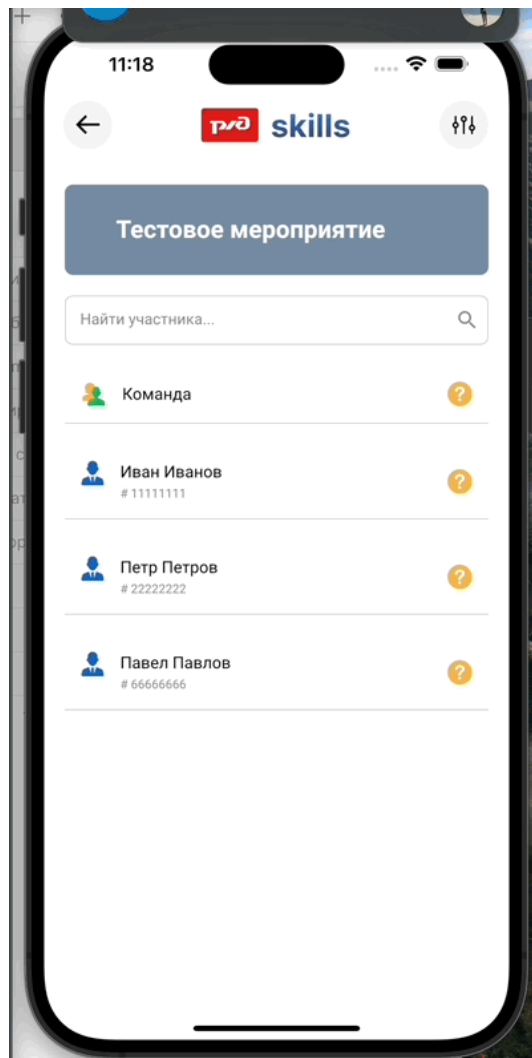
We use this method when we remove permanently like should release resource created by an object like stop animation

```
@override  
dispose() {  
    animationController.dispose(); // you need this  
    super.dispose();  
}
```



```
75     }
76 }
77
78 @override
79 void deactivate() {
80     // TODO: implement deactivate
81     print('deactivate');
82     super.deactivate();
83 }
84
85 @override
86 void dispose() {
87     // TODO: implement dispose
88     print('dispose');
89     _animationController.dispose();
90     super.dispose();
91 }
92
93 @override
94 Widget build(BuildContext context) {
95     return Scaffold(
96         appBar: AppBar(
97             title: Text(widget.title),
98             ), // AppBar
99         body: Center(
```

List Tile



List Tile

The ListTile widget is a pre-designed layout used to create a row with leading, title, subtitle, and trailing widgets. It's especially useful inside lists like ListView.

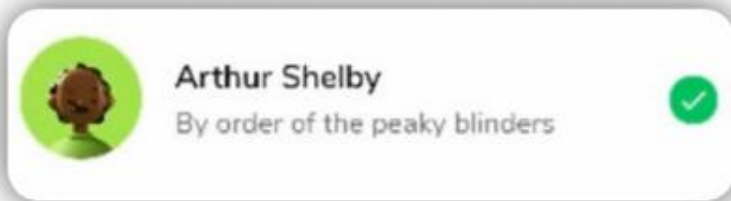
When to Use ListTile

Use ListTile when you need a simple, consistent way to display:

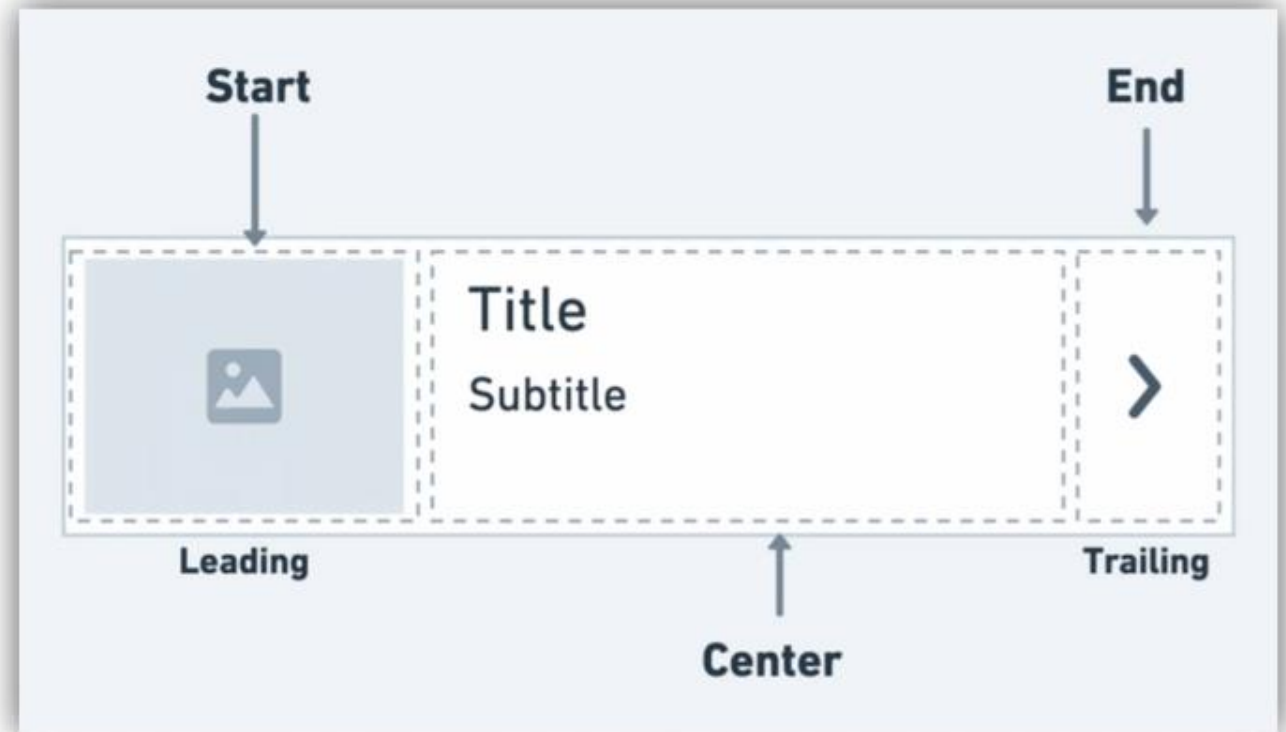
- Settings options
- Contacts list
- Message previews
- Notification items
- Menu actions

List Tile

Basics of ListTile



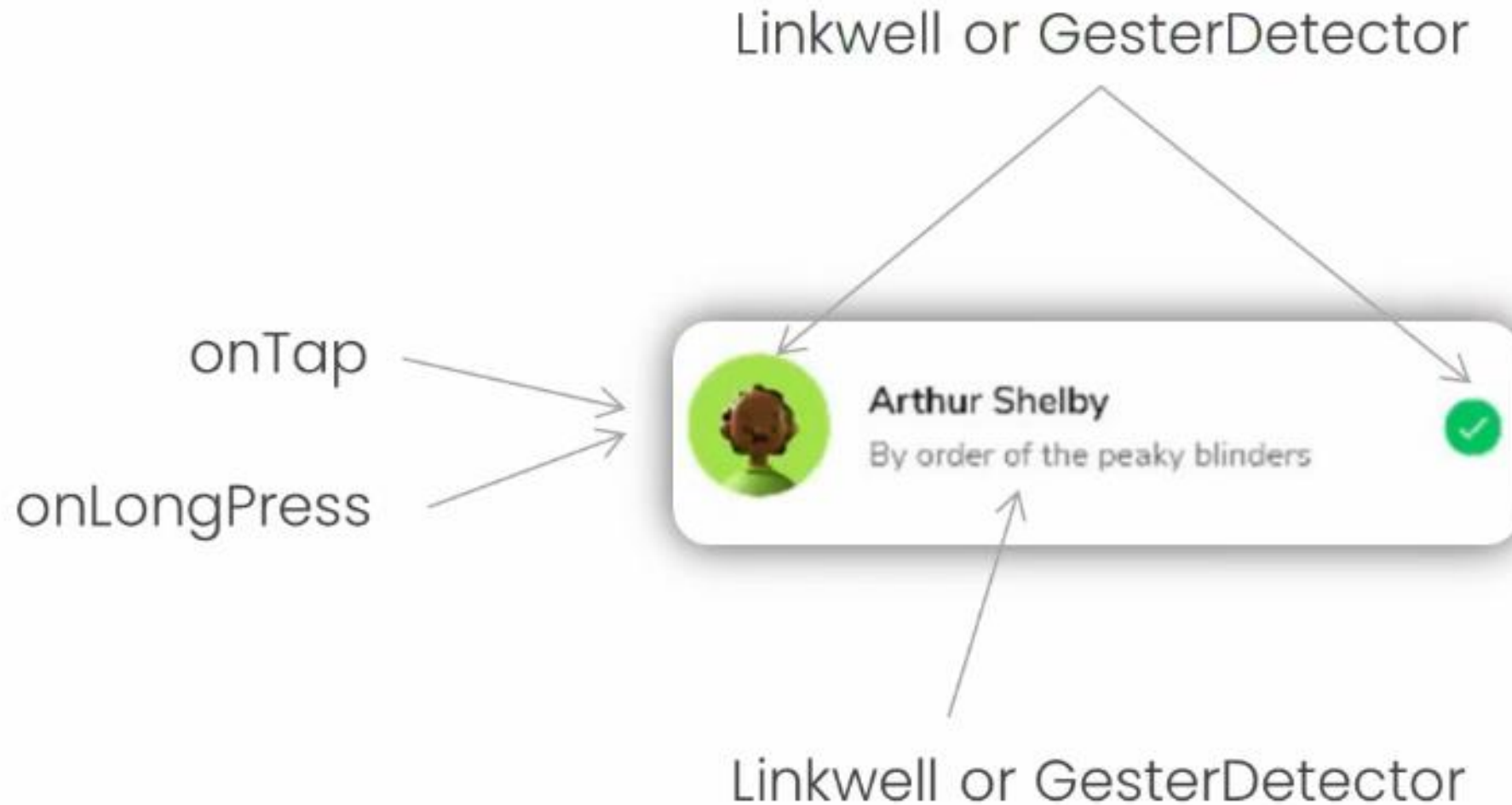
- ✓ leading
- ✓ title
- ✓ subtitle
- ✓ trailing



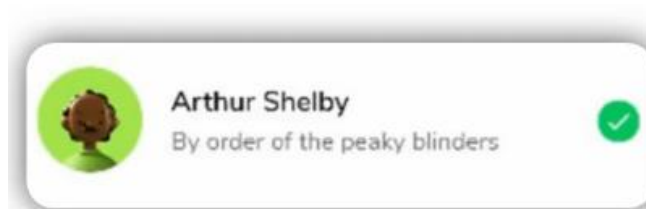
Properties of List Tile

- **title:** A widget that represents the primary content of the ListTile. Typically, this is a Text widget.
- **subtitle:** A widget that represents secondary content, usually placed below the title. Often used for additional information.
- **leading:** A widget displayed before the title. Commonly used for icons or images.
- **trailing:** A widget displayed after the title. Often used for actions like buttons or icons.
- **onTap:** A callback function that is called when the ListTile is tapped.
- **onLongPress:** A callback function that is called when the ListTile is long-pressed.

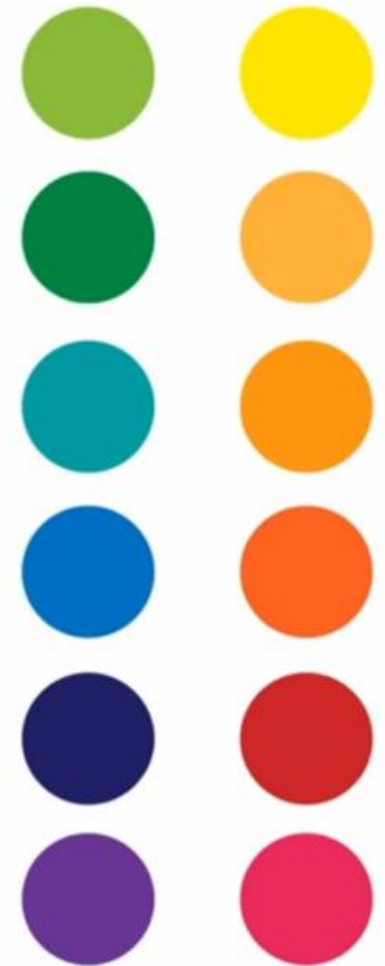
Call Back in List Tile



Color Collection in List Tile



```
iconColor: Colors.pink,  
tileColor: Colors.yellow,  
hoverColor: Colors.red,  
selectedColor: Colors.black,  
selectedTileColor: Colors.white,  
focusColor: Colors.purpleAccent,
```



REFERENCE

- <https://flutterbyexample.com/lesson/stateful-widget-lifecycle>
- <https://api.flutter.dev/flutter/material/ListTile-class.html>
- <https://api.flutter.dev/flutter/material/Card-class.html>

Thank You 😊